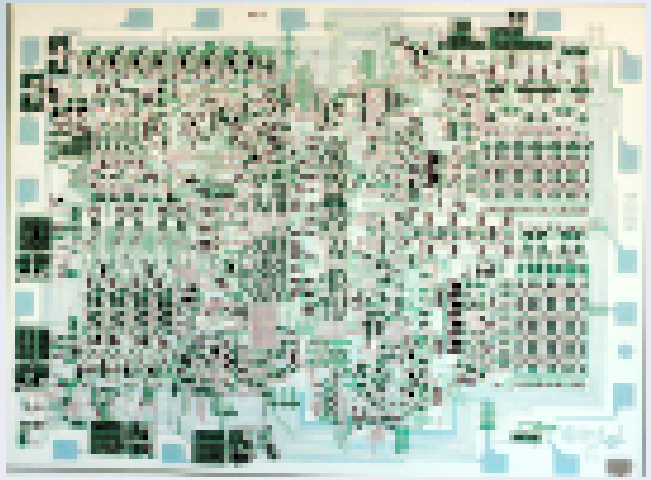

Lecture 6b: Programming the LC-3 in Assembly Language

CSCI11: **Computer Architecture and Organization**

Complete Example Program

Reads four digits and stores in memory

```
; four_digits:
; Demonstrates:
; Trap codes for I/O
; Testing for an ASCII number
; Storing in an array
; Branching based on condition codes

; Usage:
; Program to read and echo 4 numeric digits
; If input is not an ASCII number (0-9), it is ignored
; Number is saved to memory
```

```
; Registers:
; R0: Receives ASCII values
; R1: Counts number of inputs
; R2: Checks if input is a number
; R3: Stores negative ASCII values
; R4: Stores numbers in ASCII form
; Memory:

        .ORIG   x3000   ;
        BR      START

DIGITS  .BLKW   #4      ; sum of digits
```

```
        ; determine negative values of ASCII 0 and 9 for easy comparison
START   LD      R3, ASCII_0  ;
        NOT R3, R3      ;
        ADD R3, R3, #1  ; 2's complement of ASCII 0
        ST R3, ASCII_0  ; store in ASCII_0
        LD R3, ASCII_9  ;
        NOT R3, R3      ;   ;
        ADD R3, R3, #1  ; 2's complement of ASCII 0
        ST R3, ASCII_9  ; store in ASCII_9
```

```
        ; display program title and description
        LEA R0, TITLE   ;
        PUTS            ; show program title
        LD R0, CR       ;
        OUT             ; print new line
        LEA R0, DESC    ;
        PUTS            ; show description
        LD R0, CR       ;
        OUT             ; print new line
```

```
        ; read 4 numeric digits
        LD R1, COUNT    ; get count
        LEA R4, DIGITS  ; get array
INPUT   IN              ; read a character
        LD R3, ASCII_0
        ADD R2, R0, R3 ; check if < ASCII_0
        BRn INPUT       ; if so, get another character
        LD R3, ASCII_9
        ADD R2, R0, R3 ; check if > ASCII_9
        BRp INPUT       ; if so, get another character
```

```
        ; save the character, decrement count and loop
        STR R0, R4, #0  ; save the digit, L->R => Low->High
        LD R0, CR       ;
        OUT             ; print new line
        ADD R4, R4, #1  ; increment storage location
        ADD R1, R1, #-1 ; decrement count
        BRnp INPUT      ; if count is zero, exit
```

```
        ; at end, print end message and halt
        LD R0, CR       ;
        OUT             ; print new line
        LEA R0, END_M   ; load address of END_M
        PUTS            ; show end message
EXIT    HALT            ; stop the program
```

```
; Data Section
TITLE   .STRINGZ "I/O and Branching"    ; title at start of output
DESC    .STRINGZ "Enter numbers only"   ; description of program
END_M   .STRINGZ "Check DIGITS for Results" ; end of program message
CR      .FILL x000A                     ; ASCII for CR
COUNT   .FILL #04                       ; Used for counting number of inputs
ASCII_0 .FILL x0030                     ; ASCII for 0
ASCII_9 .FILL x0039                     ; ASCII for 9
        .END                            ; end of source code
```

Begin thinking about the SMC Program

Phase 1 - Register-based parameters and result in memory

1. Get (2) 4-digit ASCII numbers and convert to decimal
2. Get an operator (+, -, *, /) and call operation
3. Execute operation (+, -, *, /)
4. Save result in memory

Phase 2 - Stack-based parameters and result on console

1. Add the stack for passing parameters
2. Add RPN to allow for multiple operations
3. Add decimal to ASCII conversion to display to console

Phase 3 - Error checking and app polishing

1. Check for overflow or underflow
2. Provide guidance on errors
3. Think of the user experience (UX)

Phase 4 - Documentation

1. Create a README with install instructions
2. Add user manual as to how to use calculator
3. Document the code, extensively